Digital Integrated Circuit Design

Lecture-5: Multioperand Addition

Chih-Feng Wu (吳志峰), PhD.

Department of Electronics Engineering, Chang Gung University,
Taoyuan, Taiwan (ROC)

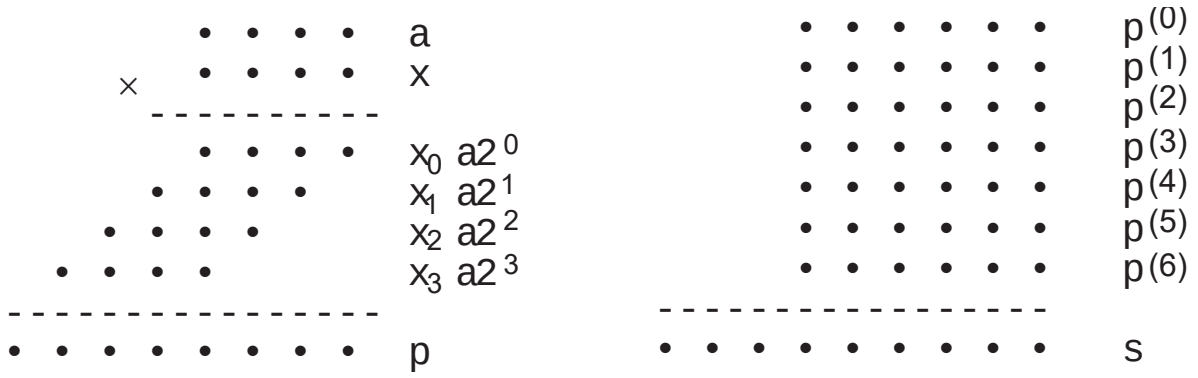
May 8, 2026

Outline

- ❑ **Using Two-Operand Adder**
- ❑ **Carry-Save Adder**
- ❑ **Wallace and Dadda Tree**
- ❑ **Parallel Counters and Compressors**

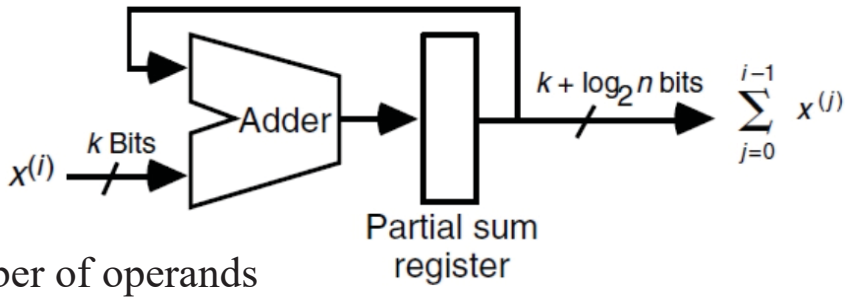
Using Two-Operand Adder

- Some applications of multioperand addition
  - Multiplication or inner-product computation in **dot** notation



Using Two-Operand Adder (cont'd)

- Serial implementation of multioperand addition with a single 2-operand adder



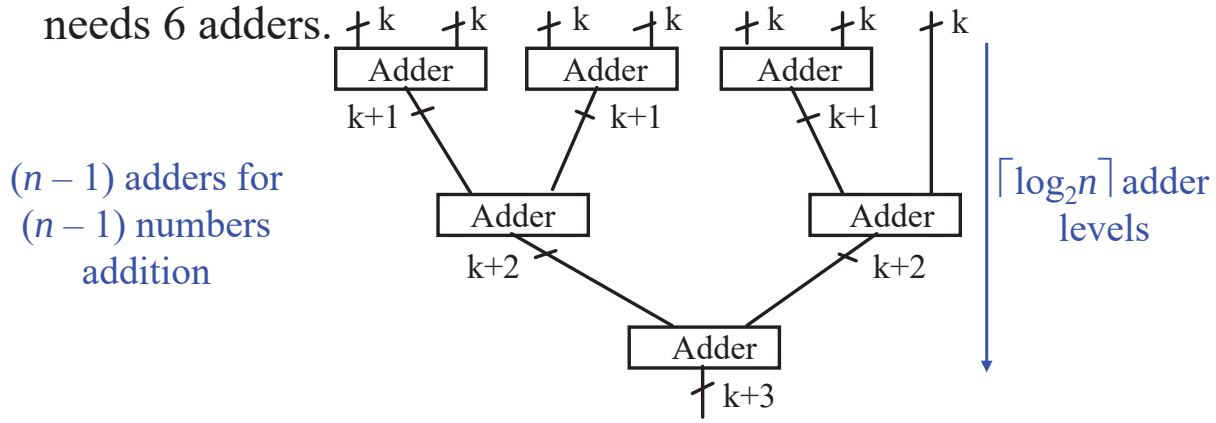
  - n: the number of operands
  - Partial sum register $= k + log_2 n$ bits
  - Assuming a logarithmic-time fast adder, the delay is derived as

$$T_{serial-multi-add} = O(n log_2[k + log_2 n])$$

  - The addition time grows super-linearly with n when k is fixed and logarithmically with k for a given n.

Parallel Implementation as Tree of Adders

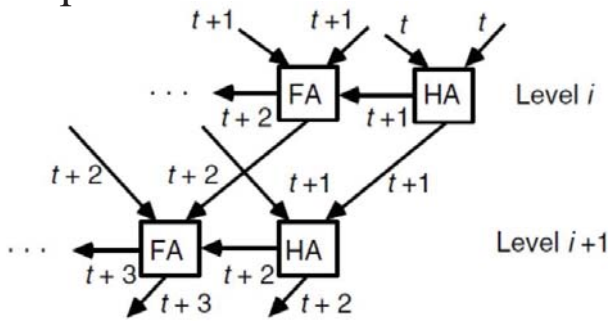
- Adding 7 numbers in a binary tree of adders, the operation needs 6 adders.



  - Assuming a logarithmic-time fast adder, the delay is derived as
    - $T_{tree-fast-multi-add}$
      $= O(log_2 k + log_2[k+1] + \cdots + log_2[k + \lceil log_2 n \rceil - 1])$
      $= O(log_2 n \cdot log_2 k + log_2 n \cdot log_2[log_2 n])$
      $= O\{log_2 n \cdot (log_2 k + log_2[log_2 n])\}$

Parallel Implementation as Tree of Adders (cont'd)

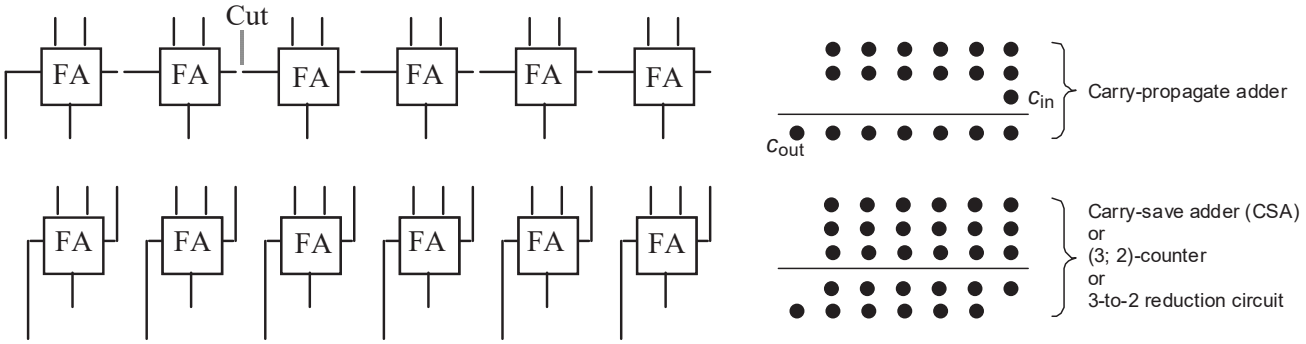
- Ripple-carry adders at levels i and $i + 1$ in the tree of adders used for multi-operand addition



  - $T_{tree-ripple-multi-add} = O(k + log_2 n) < T_{tree-fast-multi-add}$
  - The absolute best latency that we can hope for is O($log_2 k + log_2 n$)
  - There are kn data bits to process and using any set of computation elements with constant fan-in, this requires O($log_2(kn)$) time.
  - We will see shortly that carry-save adders achieve this optimum time

Carry-Save Adder

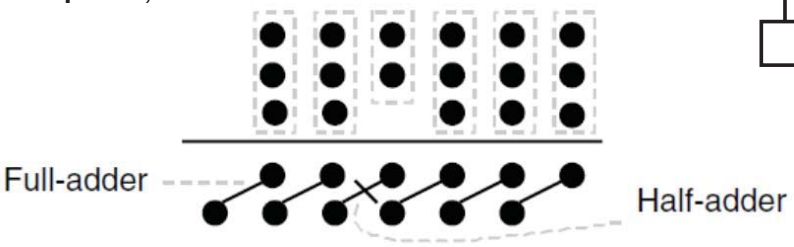
- A ripple-carry adder turns into a carry-save adder if the carries are saved (stored) rather than propagated.



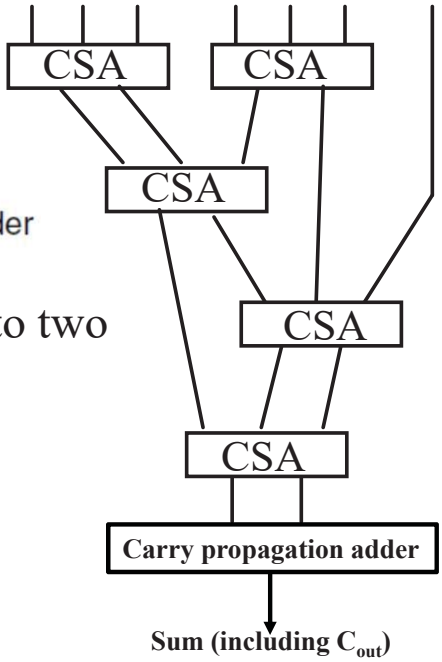
- Carry-propagate adder (CPA) and carry-save adder (CSA) functions in dot notation

Bit Aerial Adder and Ripple Carry Adder

- Specifying full- and half-adder blocks, with their inputs and outputs, in dot notation

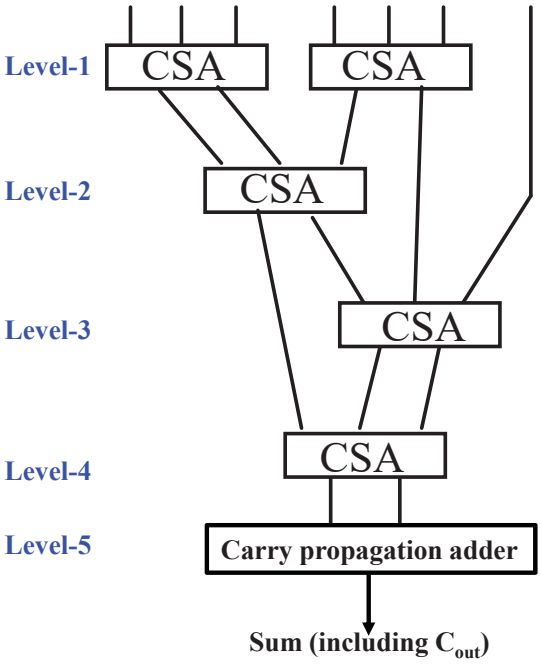


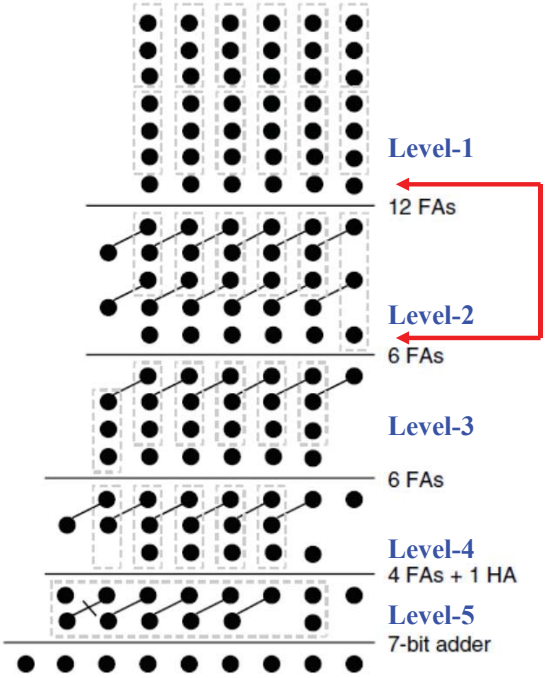
- Tree of CSAs reducing seven numbers to two
  - No. of CSA tree level (height) = $log_2 n$
  - $C_{CSA\_Multi\_add} = (n-2)C_{CSA} + C_{CPA}$
  - $T_{CSA\_Multi\_add} = O(tree\ height + T_{CPA})$
    $= O(log_2 n + log_2 k)$
- The CPA width = $k + \lceil log_2 n \rceil$



Carry-Save Adder

- Addition of seven 6-bit numbers in dot notation





Carry-Save Adder (cont'd)

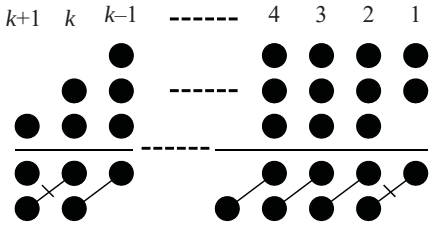
- Addition of seven 6-bit numbers in dot notation

8	7	6	5	4	3	2	1	0	Bit position
			7	7	7	7	7	7	6 × 2 = 12 FAs
		2	5	5	5	5	5	3	6 FAs
		3	4	4	4	4	4	1	6 FAs
	1	2	3	3	3	3	2	1	4 FAs + 1 HA
	2	2	2	2	2	1	2	1	7-bit adder
			Carry-propagate adder						
1	1	1	1	1	1	1	1	1	

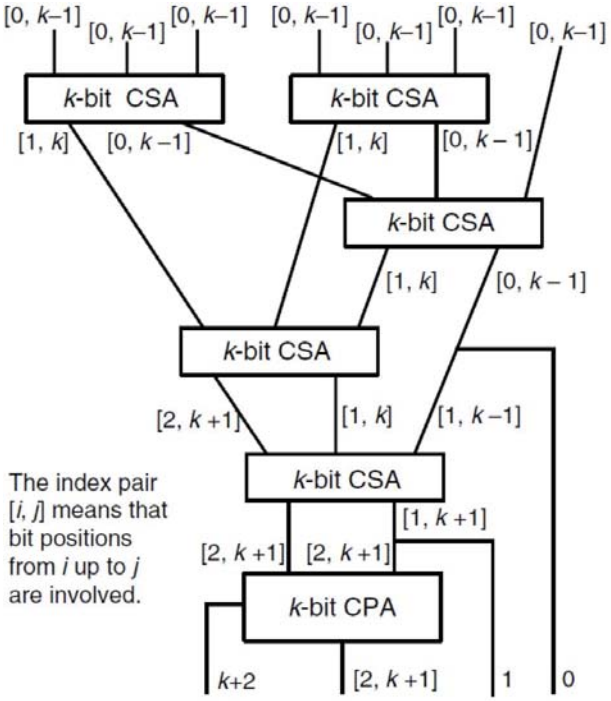
Total cost = 7-bit adder + 28 FAs + 1 HA
CPA width [k+1: 1]

Carry-Save Adder (cont'd)

- Adding seven k-bit numbers and the CSA/CPA width
  - CPA width: [____ , __]

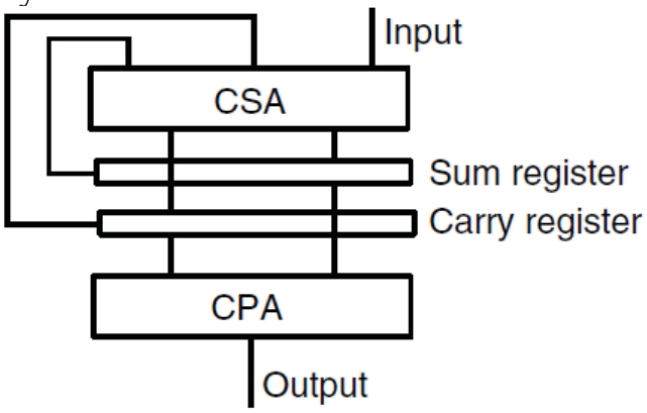


CPA width [k+1: 2]



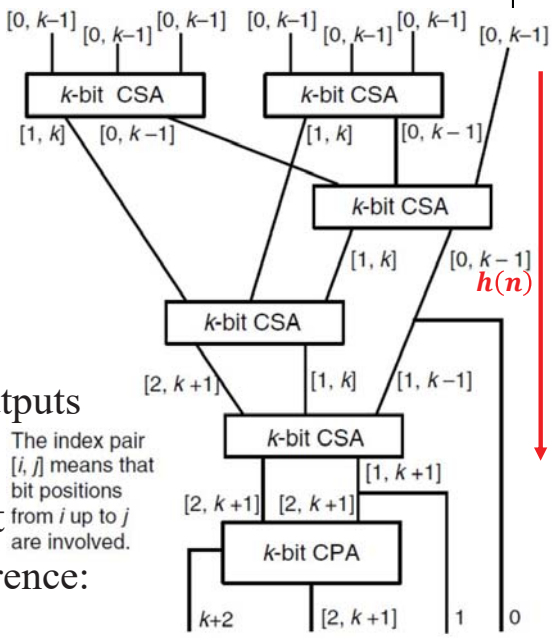
Carry-Save Adder (cont'd)

- Serial carry-save addition using a single CSA
  - Suitable for the following situation
    - The input operands arrive serially
    - Read out from memory one by one
  - Disadv.
    - Wider (bit-width) for both CSA and CPA



Wallace and DADDA Trees

- Wallace tree
  - Reduce the number of operands at the earliest possible opportunity
- Seven-input **Wallace** tree
  - Reduce k-bit inputs to two $(k+1)$-bit operands
  - k-bit inputs → $(k + log_2 n - 1)$-bit outputs



- The smallest height $h(n)$ of an n-input Wallace tree satisfies the following recurrence:

  $h(n) = 1 + h(\lceil 2n/3 \rceil)$
  - Lower bound:

    $h(n) \geq log_{1.5}(n/2)$ for n=2, 3

Wallace and DADDA Trees (cont'd)

- The recurrence for $n(h)$ (the max no. of input) for an h-level tree

  $n(h) = \lfloor 3n(h-1)/2 \rfloor$
  - Upper bound:

    $n(h) = 2(3/2)^h$
  - Lower bound:

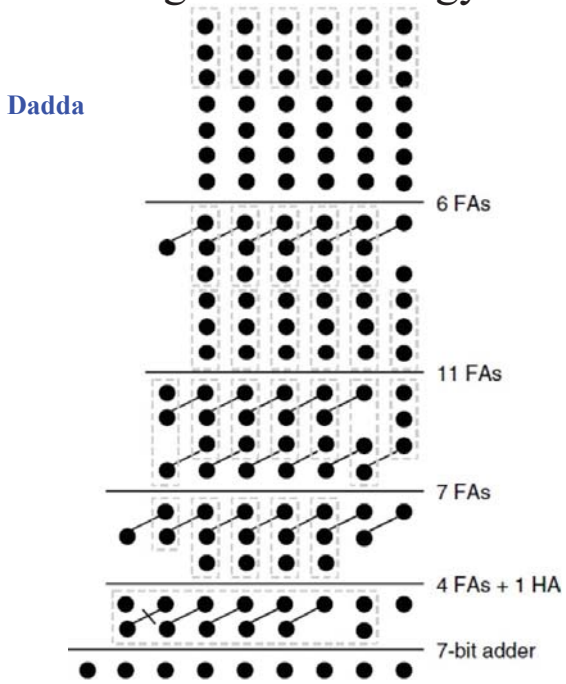
    $n(h) = 2(3/2)^{h-1}$
- For $0 \leq h \leq 20$, we have $n(h)$ as shown the right.

h	n(h)	h	n(h)	h	n(h)
0	2	7	28	14	474
1	3	8	42	15	711
2	4	9	63	16	1066
3	6	10	94	17	1599
4	9	11	141	18	2398
5	13	12	211	19	3597
6	19	13	316	20	5395

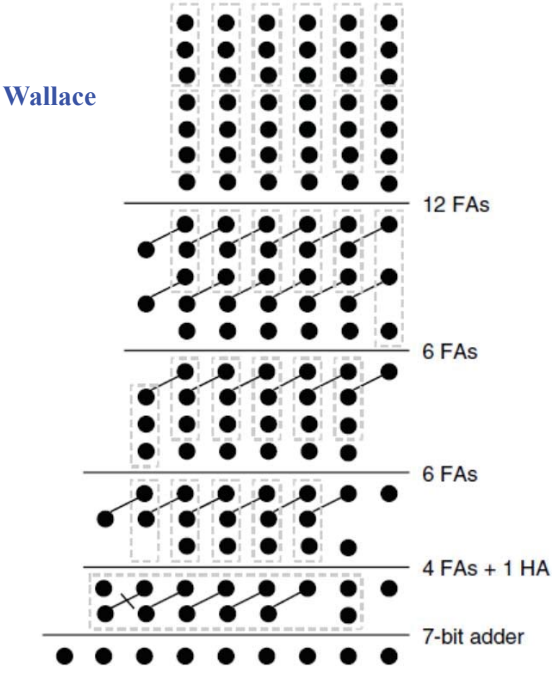
- Dadda tree
  - Postpone the reduction to the extent possible without causing added delay

Wallace and DADDA Trees (cont'd)

- Using Dadda strategy to add seven 6-bit numbers



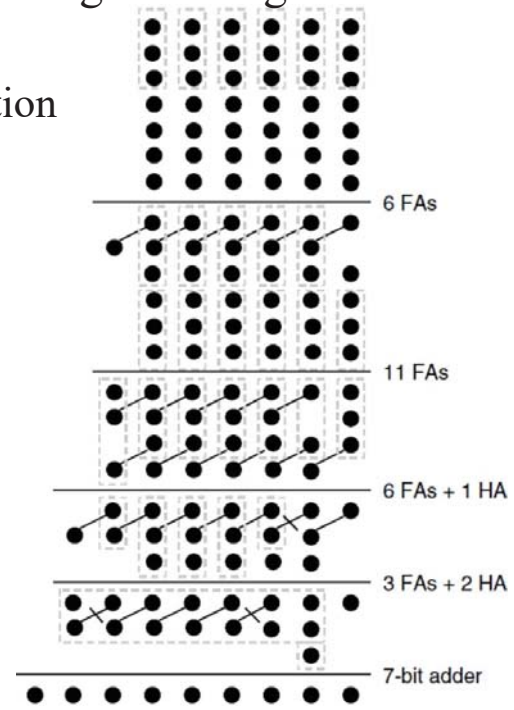
Total cost = 7-bit adder + 28 FAs + 1 HA



Total cost = 7-bit adder + 28 FAs + 1 HA

Wallace and DADDA Trees (cont'd)

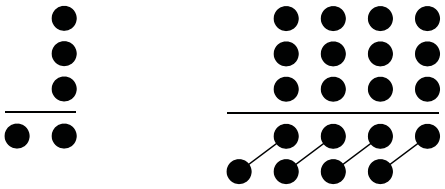
- Adding seven 6-bit numbers by taking advantage of the final adder carry-in
  - Leave three dots in the LSB position of CPA
  - Replace two FAs with HAs



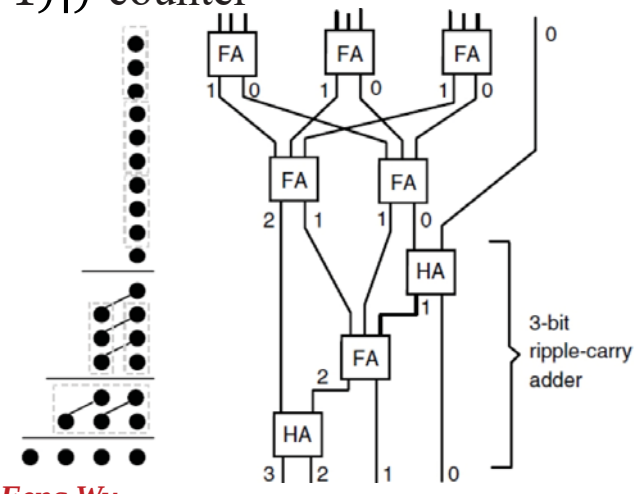
Total cost = 7-bit adder + 26 FAs + 3 HA

Parallel Counters and Compressors

- 1-bit FA → (3:2)-counter
  - 3-bit input
  - 2-bit result/output



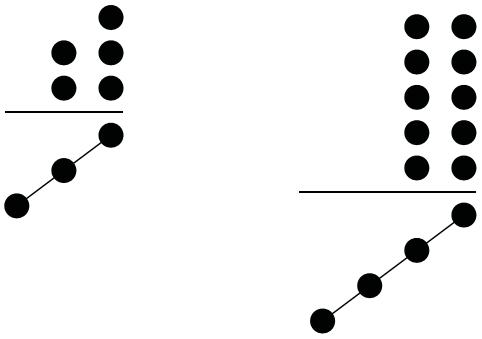
- Closed-form: $(n: \lceil log_2(n+1) \rceil)$-counter
  - n inputs
  - $\lceil log_2(n+1) \rceil$-bit output



- (10:4)-counter

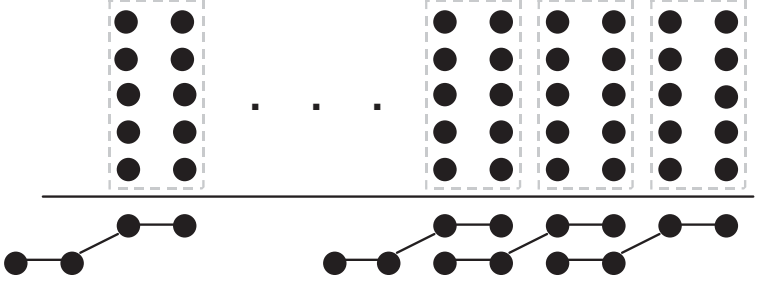
(cont'd)

- Unequal columns
  - (2, 3: 3)-counter



- Multicolumn reduction

- (5, 5; 4)-counter
  - Reduce five numbers to two numbers



- In summary
  - Generate parallel counter = Parallel compressor

Add Multiple Signed Numbers

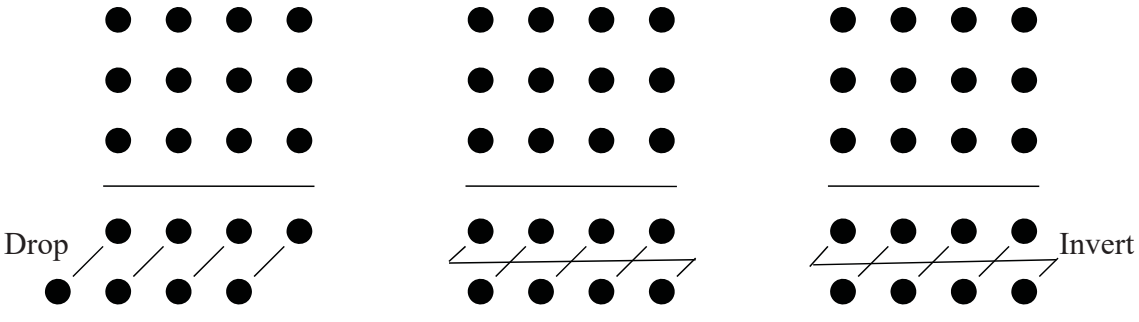
- Add multiple operands with 2's complement format
  - Sign-extension to the width of the final result

-- Extended positions -- **Sign** -- Magnitude positions ------

$$
\begin{array}{ccccc|ccccc}
x_{k-1} & x_{k-1} & x_{k-1} & x_{k-1} & x_{k-1} & \boldsymbol{x_{k-1}} & x_{k-2} & x_{k-3} & x_{k-4} & \cdots \\
y_{k-1} & y_{k-1} & y_{k-1} & y_{k-1} & y_{k-1} & \boldsymbol{y_{k-1}} & y_{k-2} & y_{k-3} & y_{k-4} & \cdots \\
z_{k-1} & z_{k-1} & z_{k-1} & z_{k-1} & z_{k-1} & \boldsymbol{z_{k-1}} & z_{k-2} & z_{k-3} & z_{k-4} & \cdots
\end{array}
$$

Modular Multioperand Adder

- Modular carry-save addition with special moduli



(a) $m = 2^k$ (b) $m = 2^k - 1$ (c) $m = 2^k + 1$

Modular Multioperand Adder (cont'd)

- Modular Reduction with pseudoresidue
  - Modulo-21 reduction of 6 numbers taking advantage of the fact that **64 = 1 mod 21** and using 6-bit pseudoresidue

If $\boldsymbol{m}$ is such that $\boldsymbol{2^h = 1}$ mod $\boldsymbol{m}$ for a fairly small value of $\boldsymbol{h}$, one can perform tree reduction with h-bit pseudoresideus and **end-around** carry.
The end-around carry is used for modulo-63.